

Documentación Técnica de Sustentación

App Store Review Analyzer — Backend

Proyecto: IA-PROYECT · Módulo: proyecto_resenas (reorganizado como backend)

Este documento describe la arquitectura, la organización de carpetas y el flujo de conexión del backend, como material de apoyo para la sustentación del proyecto.

1. Objetivo del proyecto

El proyecto implementa un backend (API REST) que analiza reseñas de usuarios escritas en español. Por cada reseña, el sistema: (1) simula una traducción al inglés, (2) calcula el número de tokens que consumiría el texto en cada idioma usando la librería **tiktoken**, y (3) extrae datos estructurados sobre el error reportado (tipo, componente afectado, severidad y un resumen en ambos idiomas). El objetivo práctico es demostrar cuántos tokens (y por lo tanto costo) se pueden ahorrar al procesar reseñas de soporte técnico en inglés en lugar de español, algo relevante cuando estas reseñas se envían a un modelo de lenguaje (LLM) para su análisis automático.

2. Arquitectura general

El sistema se compone de tres partes independientes que se comunican entre sí únicamente por HTTP, siguiendo el patrón cliente-servidor:

- **Backend (API):** aplicación FastAPI que expone el endpoint POST `/api/analyze`. Es el único componente que debe estar corriendo de forma permanente.
- **Script generador de datos:** crea un archivo Excel con reseñas ficticias, usado únicamente para tener datos de prueba.
- **Script cliente (procesador masivo):** lee el Excel y envía cada reseña al backend en paralelo (peticiones HTTP asíncronas), luego imprime métricas agregadas de tokens y costo.

No se incluye frontend: el backend se puede probar directamente desde su documentación interactiva automática (Swagger UI), desde Postman, desde la terminal con `curl`, o desde el script cliente incluido.

3. Estructura de carpetas

backend/	
app/	Backend (API)
main.py	Punto de entrada: crea la app y conecta routers
core/	
config.py	Configuración y constantes
models/	
schemas.py	Modelos Pydantic (request / response)
routers/	
analyze.py	Endpoint POST <code>/api/analyze</code>
services/	
analysis_service.py	Lógica de negocio (tokens, traducción, extracción)
scripts/	Herramientas externas al backend
generar_excel.py	Genera datos de prueba (Excel)
procesar_excel_async.py	Cliente que consume el backend en paralelo
data/	
resenas_productos_50k.xlsx	
requirements.txt	
README.md	

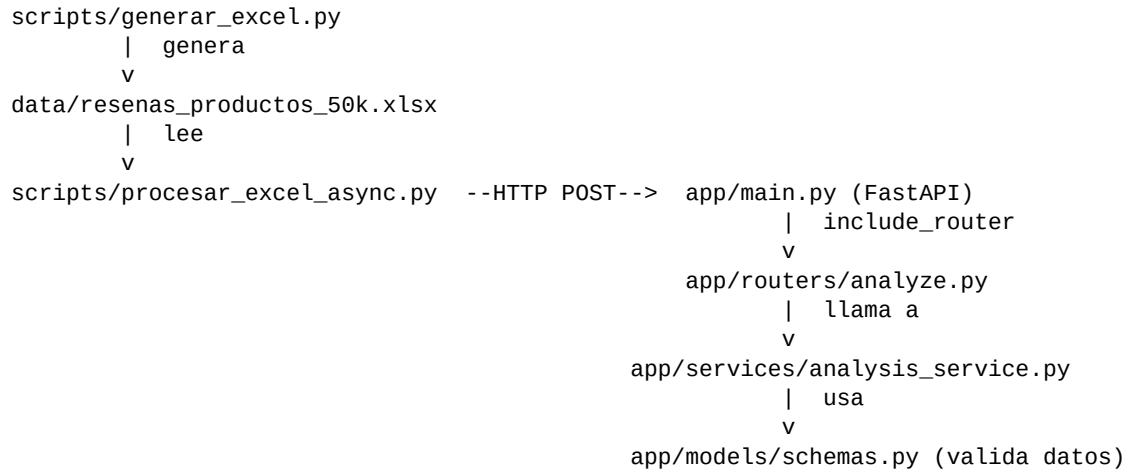
Esta organización separa responsabilidades siguiendo una arquitectura en capas típica de un backend FastAPI: **routers** (entrada HTTP), **services** (lógica de negocio) y **models** (contratos de datos), lo que facilita mantener, probar y escalar el proyecto.

4. Descripción de cada componente

Archivo	Responsabilidad
app/main.py	Crea la app FastAPI, configura CORS y conecta (include_router) los endpoints definidos en routers.
app/core/config.py	Centraliza constantes: nombre de la API, encoding de tokens, precio por millón de tokens, host/origen de datos.
app/models/schemas.py	Define los modelos Pydantic que validan la petición de entrada y dan forma a la respuesta JSON.
app/routers/analyze.py	Define el endpoint POST /api/analyze. Valida el texto recibido y delega el trabajo al servicio.
app/services/analysis_service.py	Lógica de negocio: carga el codificador tiktoken, cuenta tokens en español e inglés, simula la traducción.
scripts/generar_excel.py	Genera un Excel con reseñas ficticias en español (usando Faker), guardado en data/.
scripts/procesar_excel_async.py	Cliente HTTP asíncrono: lee el Excel y envía cada reseña al backend en paralelo para medir tiempos.

5. Flujo de conexión (cómo se comunican los componentes)

El siguiente esquema resume cómo los scripts se conectan al backend y cómo el backend enruta internamente cada petición:



Punto clave: los scripts nunca importan código del backend directamente. Se comunican exclusivamente por HTTP, tal como lo haría cualquier otro cliente (una app móvil, un frontend web o Postman). Esto es lo que permite decir que el backend es completamente independiente y reutilizable por cualquier consumidor externo.

6. Endpoint expuesto

POST /api/analyze

Petición (JSON):

```
{
  "text": "La aplicación se cierra al subir una foto de perfil."
}
```

Respuesta (JSON):

```
{
  "metrics": {
    "original_tokens": 12,
    "translated_tokens": 18,
    "tokens_saved_per_request": -6
  },
  "extracted_data": {
    "error_type": "crash",
    "component": "profile_picture_upload",
    "severity": "high",
    "summary_en": "App crashes when uploading profile picture from gallery.",
    "summary_es": "La app se cierra al subir foto de perfil desde la galería."
  }
}
```

También se expone GET / como endpoint de salud, útil para verificar que el servicio está activo antes de ejecutar pruebas masivas.

7. Instalación y ejecución

Instalación de dependencias

```
cd backend
python -m venv venv
source venv/bin/activate      # En Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Levantar el backend

```
cd backend
uvicorn app.main:app --reload
```

El backend queda disponible en <http://127.0.0.1:8000>, con documentación interactiva automática en <http://127.0.0.1:8000/docs>.

Probar con datos masivos (opcional)

```
# 1) Generar datos de prueba
python scripts/generar_excel.py

# 2) Con el backend ya corriendo en otra terminal:
python scripts/procesar_excel_async.py
```

8. Conclusiones

La reorganización en capas (routers, services, models, core) separa claramente la entrada HTTP de la lógica de negocio y de los contratos de datos, lo que facilita explicar, mantener y ampliar el proyecto. El backend queda desacoplado de cualquier cliente: puede ser consumido por el script de pruebas incluido, por Postman/cURL, o en el futuro por un frontend, sin necesidad de modificar su código interno. Los puntos de extensión más claros son la función de traducción y la de extracción de datos del error, hoy simuladas, que podrían reemplazarse por un servicio de traducción real o un modelo de lenguaje sin afectar al resto de la aplicación.